Retrieval, measured

A first-principles walkthrough of the retrieval-ablation project

---

Part I - The problem, from nothing

1. What this project actually is

Imagine you have 120 annual reports from large American companies. Together they run to about 27,500 pages. Someone asks:

> What was Apple's research and development expense in fiscal 2025?

The answer is one number, sitting in one row of one table, in one of those 120 documents. A system that answers this question has to do two separate jobs:

1. **Find** the handful of paragraphs, out of tens of thousands, that contain the answer.
2. **Write** a reply based on what it found.

Almost every published project that does this measures only the second job. It shows you an answer and asks whether the answer looks right. That is a mistake, and understanding why is the whole point of this project.

If step 1 handed step 2 the wrong pages, then step 2 is being asked to write an answer from material that does not contain one. It will either refuse, or invent something. No amount of improving step 2 fixes that. The two jobs fail for completely different reasons and are repaired by completely different means, so measuring them together tells you almost nothing about what to do next.

This project measures step 1 on its own. That is the entire idea. Everything else follows from it.

2. Why "measuring step 1 on its own" is harder than it sounds

To score the finding step by itself, you need to already know the right answer - not the answer *text*, but which specific passage contains it. You need a list like:

question	the passage that answers it
"Apple R&D expense 2025?"	document aapl-10-k-2025, characters 1,215,596-1,216,693

That list is called a **labelled evaluation set**, and building one is most of the work. With it, you can run any finding-system over the questions, see where it ranked the correct passage, and score it - **with no language model involved at all**. That makes the measurement free, instant, and perfectly repeatable.

Without it, you are guessing.

3. How text gets found: the two families

There are two fundamentally different ways to find text.

Lexical search - matching words

The classic method. You build an index of which words appear in which passages, then for a query you look

up passages containing the query's words and rank them.

The standard scoring function is **BM25**. Three ideas in it:

1. **Rare words matter more.** A passage containing "amortisation" tells you more than one containing "the". This is *inverse document frequency* - a word in few documents gets a high weight.
2. **Repetition helps, with diminishing returns.** A passage using "amortisation" five times is more likely to be about it than one using it once - but twenty times is not four times better than five. This is *term-frequency saturation*.
3. **Long passages need discounting.** A very long passage contains more words by accident, so it would win by length alone without a correction. This is *length normalisation*.

BM25 is old, simple, fast, and remains a genuinely strong baseline. It has one fundamental limitation: it can only match words that are actually there. Ask about "R&D spending" when the filing says "research and development expense" and BM25 sees almost nothing in common.

Dense search - matching meaning

The modern method. A neural network reads a passage and emits a list of numbers - say 1,024 of them - called an **embedding** or **vector**. The network is trained so that passages meaning similar things get similar lists. You embed every passage once, then embed the query, then find the passages whose vectors point in most nearly the same direction.

"Direction" is literal. Each vector is an arrow in 1,024-dimensional space. Similarity is measured by the angle between two arrows - the **cosine similarity**. Identical direction scores 1.0, perpendicular scores 0.0.

Dense search handles "R&D spending" versus "research and development expense" easily, because the network learned they mean the same thing. Its weakness is the mirror image of BM25's: it is fuzzy. Ask for a specific number, or a specific year, and a dense retriever will happily return something that is *about* the right topic but concerns the wrong fiscal year.

Hybrid - using both

Since the two fail differently, combining them should help. The difficulty is that their scores are not comparable: BM25 produces unbounded sums of word weights, cosine similarity lives between -1 and 1. Adding them is meaningless.

The standard fix is **Reciprocal Rank Fusion (RRF)**. Throw away the scores entirely and keep only the *positions*. A passage ranked 1st by one system and 3rd by the other gets `1/(k+1) + 1/(k+3)`, with `k` a constant (60 by convention). Sum across systems, sort. Because only ranks are used, the incomparable-scores problem simply disappears.

The cost of discarding scores is real: RRF cannot tell a confident first place from a marginal one.

Reranking - the expensive second look

All the above are **bi-encoders**: query and passage are processed separately and compared at the end. That is fast - passages are embedded once, in advance - but the model never sees the query and passage *together*, so it cannot reason about how they relate.

A **cross-encoder** reads the query and one passage jointly and outputs a relevance score directly. Much more accurate, and far more expensive: one full neural network run per passage, with nothing precomputable.

So cross-encoders are used as a **second stage**. A cheap retriever produces a shortlist of, say, 50 candidates; the cross-encoder rescores just those and reorders them. This is widely described as the highest-value component in production retrieval. Whether that is true *on a given corpus* is exactly the sort of claim this project exists to check rather than repeat.

4. How finding is scored

Three measurements, each answering a different question.

Recall@50 - "did it find the answer at all?"

Of the passages that are genuinely relevant, what fraction appear in the top 50? Simple, and it measures a **ceiling**: anything not in the top 50 cannot be used by a later stage, no matter how clever that stage is.

MRR - "how far down was the first good hit?"

Mean Reciprocal Rank. If the first relevant passage is at position 1, score 1.0. Position 2 scores 0.5, position 4 scores 0.25. Averaged over queries. It cares only about the first correct hit.

nDCG@10 - "is the top of the list good?"

Normalised Discounted Cumulative Gain, the standard in information retrieval. Built in three steps:

1. **Gain.** Each passage has a relevance grade. A grade of 2 contributes `2^2 - 1 = 3`; a grade of 1 contributes $2^1 - 1 = 1$; irrelevant contributes 0.
2. **Discount.** A hit at position `i` is divided by `log2(i + 1)`. Position 1 is worth full value, position 10 about a third. Sum these to get **DCG**.
3. **Normalise.** Compute the DCG of the *perfect* ranking - the **IDCG** - and divide. The result sits between 0 and 1.

Step 3 hides a trap that this project cares about a great deal. The ideal ranking must be built from the *complete* list of known-relevant passages, not from the passages the system happened to return. If you normalise by what came back, a system returning one relevant passage and nothing else scores a perfect 1.0 - because that single hit is also the best possible ordering *of that list*. The metric would then reward returning less. There is a regression test in this project named after that failure.

5. The other way: just read everything

Modern language models accept enormous inputs - the one used here takes 1,048,576 tokens, roughly 800,000 words. So why retrieve at all? Why not paste the whole filing into the prompt?

This is the **long-context** approach and it is a genuine alternative, so this project measures it head to head. The trade-offs:

- **Cost.** You pay per token of input. A retrieval prompt is a few thousand tokens; a whole filing is over a hundred thousand.
- **Latency.** More input takes longer to process.
- **Accuracy.** Models are known to lose track of information buried in the middle of very long inputs.

- **Citations.** If the model got one enormous blob, it cannot tell you *which part* it used. Retrieval knows exactly which passages it supplied.

Part II - What was built

6. Choosing a corpus

The corpus is **120 SEC 10-K filings: 30 companies x 4 consecutive fiscal years** - 68.8 million characters, 14,537 extracted tables.

Four deliberate choices:

Annual reports, not Wikipedia. Filings are dense, table-heavy, full of cross-references ("see Note 12"), and structured by a mandated Item 1-15 hierarchy. Table extraction is where naive pipelines visibly fail.

Four years per company, not one. This is the most important corpus decision. Consecutive annual reports repeat their structure almost verbatim while the figures change. So "R&D expense in fiscal 2025" has three near-identical distractors differing only in the numbers. Retrieval must discriminate on the *year*. A single-year corpus would make the task far too easy and every configuration would score alike.

HTML, not PDF. Filings are *born* as HTML with explicit `<tr>`/`<td>` structure including colspan and rowspan. Converting to PDF and running a layout model throws that away and forces the model to re-infer cell boundaries from pixel positions - reintroducing error into exactly the thing being measured.

A committed ticker list, not a query. "The 30 largest US companies" resolves differently every quarter, so results against it could never be reproduced. The tickers are hardcoded; CIK identifiers are resolved from SEC's official mapping at build time.

7. The central design decision

> **Gold labels anchor to character spans in the document's canonical text, never to chunk identifiers.**

Here is why this is the load-bearing decision in the entire project.

Retrieval does not operate on whole documents - a 500,000-character filing is far too big to hand to anything. Documents are split into **chunks** of a few hundred words each. And *how* to chunk is one of the things being measured, so the chunker changes between configurations.

Now: if a gold label said "chunk 47 of document X is the answer", then switching from fixed-size to structure-aware chunking would renumber every chunk. Chunk 47 would be a completely different piece of text. The evaluation set would silently become wrong, and - this is the dangerous part - **every number would still look plausible.** Nothing would error. You would publish a table comparing systems against different ground truth while believing they shared it.

So labels record character offsets into the document's canonical text, which no chunker is permitted to alter. A chunk is judged relevant if it *covers* the gold span. Consequences:

- The same eval set scores every chunking configuration without modification.
- Adding a new chunker later requires no relabelling.
- The canonical text is immutable. All normalisation happens once, during parsing, *before* any offset is assigned.

The trap that follows from it

Coverage needs a threshold - a chunk counts as relevant if it contains at least 50% of the gold span. But what if a gold span is *longer* than any chunk a configuration produces?

Then no chunk reaches the threshold, the query has no relevant unit, and it is **silently dropped** from that configuration's average.

That is a measurement disaster wearing the clothes of a reasonable default. Small-chunk configurations would drop every query whose answer is a long table - precisely the queries they fail hardest at - while structure-aware chunking keeps them. The two would be compared on *different query sets*, and small chunks would look better than they are.

Two defences, both implemented: **reachability** is reported per configuration (what fraction of gold passages it can represent at all), and the headline comparison runs on the **intersection** of queries every configuration can score.

8. Module by module

```
corpus/     fetching and parsing filings into canonical text + structure
chunking/   three strategies for splitting documents
evalset/    building, checking, and applying the labelled benchmark
index/      BM25, dense, fusion, reranking
metrics/    nDCG / Recall / MRR, and the statistics for reporting them
llm/        cached, quota-tolerant access to a language model
generation/ answering questions and scoring the answers
ablation/   the experiment grid and its runner
service/    FastAPI API and the inspection UI
```

`corpus/models.py` - the canonical `Document`, its `Blocks` (paragraph, heading, table, boilerplate), and the `Span` algebra. Span coverage is deliberately **asymmetric**: it asks "how much of the gold passage does this chunk contain?", not "how similar are these two spans?". A symmetric measure like Jaccard would penalise a large chunk that fully contains a short answer, which is a *successful* retrieval.

`corpus/edgar.py` - a polite, cached, resumable SEC client. Rate-limited to well under SEC's published ceiling, with a User-Agent naming a real contact (verified necessary: a generic `python-httpx/0.27` User-Agent is refused with HTTP 403). Every fetch is cached, so re-running costs no network traffic.

`corpus/html_parse.py` - the hardest module. Turns filing HTML into canonical text with tables preserved. Discussed at length in Part III, because most of it was arrived at by being wrong first.

`chunking/` - three strategies behind one interface. *Fixed-size* packs words to a token budget with overlap, ignoring structure. *Structure-aware* never splits a table and never crosses a section boundary. *Semantic* embeds each sentence and breaks where consecutive meanings diverge.

`index/bm25.py` - BM25 written by hand rather than taken from a library, for two reasons. The tokeniser has to understand money: a filing writes `34,550` and a question might write `34550`, and the usual lowercase-and-split treats those as unrelated strings on a corpus whose answers are almost all numbers. And the lexical arm is the baseline the headline claim is measured against - a weak or misconfigured BM25 would manufacture the conclusion that hybrid retrieval wins.

`index/dense.py` - exact brute-force cosine search, deliberately *not* approximate. An approximate index has its own recall error, and that error lands inside the number the ablation is trying to read. A configuration could look better or worse because of how the graph was built, and no amount of repetition would separate that from

a real retrieval difference. At 42,215 chunks the exact matrix is 152 MB and a query is one matrix-vector product.

`metrics/stats.py` - the module that turned out to matter most. A 15-row table of bare point estimates is not evidence. Two choices:

- A **paired randomisation test**, because both systems see identical queries and pairing removes query difficulty from the variance. Smucker, Allan & Carterette (CIKM 2007) treat it as the reference method for IR.
- **Holm-Bonferroni correction** across the whole family, because comparing 14 configurations against one baseline at alpha=0.05 carries roughly a 51% chance of at least one false positive.

Part III - Decisions that turned out wrong

This is the most useful section. Every item is a real mistake made during construction, how it surfaced, and what replaced it.

9. Parsing: five wrong turns

9.1 Feeding the parser a decoded string

The first parser took `str`. The very first real filing failed: inline-XBRL filings are XHTML carrying their own `<?xml encoding=...?>` declaration, and lxml refuses a `str` that declares an encoding - rightly, because the string has already been decoded by somebody's guess.

Worse than the crash was what would have happened without it: decoding first overrides the filing's own declaration and mojibakes the typographic characters filings are full of.

Fix: bytes travel from socket to cache to parser, so exactly one component honours the declaration. Undeclared bytes get an explicit UTF-8 parser, because lxml's fallback is a legacy single-byte encoding that turned "Café" into "CafÃ(c)".

9.2 Assuming header rows contain no digits

Header detection assumed a header row has no numbers in it. Reasonable-sounding, and wrong on exactly the tables that matter: **the headers of a financial statement are years.** Every income statement was classified as having zero header rows, stripping the column labels that the row-sentence rendering depends on and leaving bare numbers with nothing to match a query against.

Fix: a header row is one whose *stub cell is empty*. Filings put the row label in column zero and leave it blank while column headers are declared. Close to universal, and it survives multi-level headers.

9.3 Rendering tables straight from the parsed grid

Filings *lay tables out* rather than tabulating them. A three-column financial table arrives as thirty physical columns: currency symbols, percent signs, and sub-1%-width spacer columns each occupy their own cell. Rendered naively this produced rows like `| R&D | $ | 34,550 | | | | 10 | % |` and header rows with the same value repeated fifteen times.

Fix: drop columns empty across all *data* rows - judging emptiness on all rows keeps every spacer alive, because a spanning header is repeated across the spacers it covers - then merge colspan groups back into single cells.

9.4 Collapsing repeated cells by matching text

The obvious way to undo colspan smearing is to collapse adjacent cells with equal text. It fixes a row label spread across three columns - and **silently deletes one of two fiscal-year columns that happen to hold the same figure.**

Fix: record which source `<td>` produced each physical column, and collapse on shared *origin* rather than shared text. Same origin means one cell was stretched; different origins mean two cells coincidentally agree. Exact rather than heuristic.

A related bug: the fold initially skipped empty cells, which left header rows three columns wider than their data rows, so every column label lined up against the wrong value.

9.5 Suppressing the table of contents one heading at a time

A 10-K lists every heading in a contents block, then repeats them in the body. Naive heading detection finds each twice.

First attempt: suppress any heading followed by little text. This deleted `PART I` (always immediately followed by `Item 1`, so the gap is tiny by construction), one-line sections like `Item 4. Mine Safety Disclosures`, and short-but-real ones like `Item 2. Properties`. Losing the Part headings meant *no passage in the corpus carried a Part at all.*

Second attempt: require a *run* of tightly packed headings. Better, but the run does not stop at the end of the contents block - it continues into the body and swallows the first real `PART I` and `Item 1`.

Fix: require both a tight run *and* that the heading's text appears again later. That is what a table of contents *is*. The final occurrence - the body heading - is always kept.

10. The corpus was 15% garbage

Every chunker reported a maximum chunk of **131,551 tokens**. For a 512-token target that is only possible if a single whitespace-free "word" is half a megabyte long. It was.

Inline-XBRL filings carry a machine-readable payload inside a `display:none` wrapper. In one filing it was **526,296 characters - 31.7% of that document** - entirely taxonomy URIs, context identifiers and period dates. Across the corpus, **12.2 million characters, 15% of everything ingested.**

The fix targets `ix:header`. What matters is what is deliberately *not* removed: `ix:nonfraction` and `ix:nonnumeric` **wrap the visible content** - one filing had 11,042 of them, one around every reported figure. Dropping inline XBRL wholesale would have deleted every number in the corpus while leaving the prose intact.

Longest "word" after the fix: **62 characters**, down from 526,205.

11. Two corpus gaps that failed silently

Neither raised an exception. Both were found by counting documents per company.

Pagination. SEC's `filings.recent` holds only a company's most recent 1,000 filings. JPMorgan's window contained 25,601 filings covering barely one calendar year and exactly *one* annual report, with 68 further pages unread. The four banks contributed one document each instead of four.

Successor entities. The ticker-to-CIK map returns the *current* registrant. After a reincorporation that is a successor with no filing history. `XOM` resolves to a CIK holding no annual reports at all; every ExxonMobil 10-K sits under CIK

34088. BlackRock's history is split across two CIKs.

Overrides are listed explicitly rather than inferred from former names - guessing a predecessor CIK is exactly how a corpus ends up quietly holding a *different company's* filings, which is far worse than a recorded gap.

12. An experimental axis that was silently inert

The table-rendering configuration reported nDCG@10 **identical to its markdown twin to four decimal places, with the same chunk count.**

Rendering is applied during *parsing*, and the runner only threaded it through the index key - so the row-sentence configuration silently reused the markdown corpus. A plausible number for an experiment that never ran. This is precisely the class of failure the project is built to catch, and it took reading the output rather than trusting it.

The fix exposed a genuine collision: rendering changes the canonical text, so it changes every character offset, so gold spans built against one rendering do not apply to another. Resolved by re-parsing from cached bytes and re-deriving gold spans - which works only because query IDs are content-addressed on (document, row label, period) rather than on offsets.

13. The lenient-labels fix that was backwards

The IR literature on **pooling bias** warns that nDCG treats unjudged retrieved documents as non-relevant. This eval set labels exactly one gold row per query, while a filing states the same figure in the income statement, the MD&A prose, and often a segment table.

Measured on the 216-query eval set that existed when this analysis ran, and not re-run since it grew to 586 -- so read these as the effect's size, not as current figures: **15.3%** of queries have an unjudged top-10 chunk carrying the answer; **11.6%** are scored as complete misses *despite the answer being retrieved*.

So a "lenient" judgement set was built - any chunk of the gold document containing the figure counts - expecting it to bound the true score from *above*.

It does the opposite. Lenient nDCG@10 fell from 0.1912 to 0.1830; Recall@50 from 0.5324 to 0.3623.

That is arithmetic, not a retrieval result, and it traces directly back to §4: IDCG is computed from the *complete* judgement set. Widening the judgements adds relevant documents the retriever mostly did not return, so the ideal ranking improves while the actual one barely moves. Recall falls because its denominator grew. Worked through, with the gold at rank 3 of 5 and four duplicates present but not retrieved:

```
strict    nDCG@10 = 0.5000   Recall = 1.0000   MRR = 0.3333
lenient   nDCG@10 = 0.3031   Recall = 0.2000   MRR = 0.3333
```

Only MRR is a valid signal here, because it depends solely on the rank of the first relevant document and cannot be diluted by relevant documents never returned. Under lenient judgements MRR rises 0.1745 -> 0.2396, and *that* is the evidence the strict labels under-credit retrieval.

Publishing lenient nDCG as an upper bound would have been a real methodological error, reached by reasoning that sounded correct. Four assertions now pin the arithmetic so nobody "fixes" it back.

14. Infrastructure decisions that were wrong

Dependency floors set to whatever the dev machine had. `numpy>=2.1` would have forced pip to upgrade numpy inside a Kaggle session, breaking the ABI of the preinstalled torch - surfacing much later as an unrelated import error, after half an hour of corpus rebuilding. The code only needs numpy 1.17-era APIs.

Four unused runtime dependencies. `qdrant-client`, `fastapi`, `uvicorn`, `jinja2` were declared required and imported nowhere, dragging grpcio, protobuf and uvloop into every environment.

A wrapper that broke on a version pairing. `sentence-transformers` 5.x hands the tokenizer's whole `BatchEncoding` to the model as positional `input_ids`; `XLMRobertaModel.forward` then evaluates `input_ids.device`, a dict lookup that misses, and raises a **bare `AttributeError` with no message** twelve frames down. It failed on a T4 after 19 minutes of successful embedding. Fixed by calling transformers directly - BAAI's own documented usage - removing the layer rather than pinning around it.

Positional alignment of precomputed vectors. One filing parsed 360 characters longer on the GPU worker than locally. Because chunk IDs encode character spans, one chunk got a different ID. Positional alignment matched 42,214 of 42,215 and would have shifted every subsequent row by a document boundary - assigning one company's vectors to another's chunks, silently. Fixed by aligning on ID.

And the explanation for that drift was wrong. It was recorded here, in the README, and in a commit message as "SEC re-posted the filing" - a plausible story, since EDGAR does re-post filings, and one that was never checked. Re-fetching both disagreeing documents showed their raw bytes byte-identical to the committed manifest, last modified in 2023. Nothing upstream had changed. **The parser was not deterministic across machines**, and two independent causes were producing it:

- Microsoft's filing writes `&#149;` five times as a list bullet. HTML5 says numeric references in `0x80-0x9F` are reinterpreted through Windows-1252, so `&#149;` means `-`. Whether libxml2 does this depends on its version; older builds hand back `U+0095`, a meaningless C1 control character. Five characters out of 357,277 - the document length never changed, so only the digest moved.
- Southern's filing is 19.6 MB, and libxml2 enforces an internal size ceiling. When it trips it does not raise: it stops adding nodes and returns a tree that looks complete. The filing lost its closing paragraph and eleven elements, and the ceiling differs between libxml2 releases - hence two machines, two corpora.

Both are now handled explicitly rather than inherited from whatever libxml2 is installed: the C1 range is mapped in `normalize_text`, chosen because it is the one place all text passes through *before* any offset is assigned, and every parse sets `huge_tree`. After rebuilding, the local corpus reproduces the GPU worker's output exactly and the vector files align with zero orphans.

The lesson is not about libxml2. A checksum told me two documents disagreed, and I explained the disagreement instead of investigating it. The explanation was consistent with every fact I had, cost nothing to believe, and sent me looking in the wrong place - at EDGAR, which was blameless - while a reproducibility bug sat in the parser. Re-fetching the bytes took two minutes and falsified it outright. A cheap test that could have contradicted the story was available the whole time, and the story's plausibility is exactly why it did not occur to me to run it.

There is a second-order trap here too. The first rebuild after the fix reported *zero* documents changed, which looked like a refutation of the diagnosis. It was not: `ingest()` caches parsed documents in `data/interim/` and skips re-parsing, so a parser fix does nothing until that cache is cleared. A fix that appears to have no effect is not evidence the diagnosis was wrong until you have checked that the fix actually ran.

A self-referential integrity check. That drift exposed a hole in a safeguard this project *advertised*: `ingest()` writes the manifest and `load_corpus()` verifies against whatever manifest is on disk. A fresh ingest followed by `load_corpus` verified a run against its own output. The GPU worker printed "corpus verified: 120 documents" while holding a document that differed from the committed one.

And the fix for it was half a fix - which is the more useful lesson. The worker was changed to read the committed manifest into a variable *before* calling `ingest()`, and that was recorded as done. It was not: the snapshot was read, printed as a document count, and never compared to anything. The self-referential check downstream was untouched and still always passed. This survived until the vector files were audited chunk id by chunk id against a fresh local rebuild, which is when the one stale id turned up - the drift had been shipped, unnoticed, in an artifact the repository presents as verified.

Two things are worth taking from it. First, a safeguard that cannot fail is indistinguishable from no safeguard, and reading the right value is not the same as checking it; the code *looked* like a verification because a variable named `committed` appeared next to a print statement. Second, the thing that actually contained the damage was not a check at all but a representation choice - keying vectors by chunk id meant the mismatch had somewhere to show up as a count. Defences that make a whole class of error *structurally visible* outrank defences that test for it, because the test is only as good as the last person's attention and the structure holds regardless.

Query vectors reused across a rewrite of the query. Paraphrasing keeps `query_id` deliberately - that is what makes the two eval sets comparable. The artifact loader keyed vectors by id and then filed each row under whatever text the caller currently held, so the paraphrased run scored every dense arm with vectors embedded from the *original* wording. No exception, complete coverage, entirely ordinary-looking metrics. The only symptom was nDCG@10 matching the original run at four decimal places, which is the sole reason it was caught.

The artifacts now record the text each vector was built from, rows whose text has changed are dropped, and an artifact that cannot say what it embedded is refused outright. An unmeasured arm costs a re-run; an arm silently scored against stale vectors costs the credibility of every number printed beside it.

Worth naming the pattern, because this project produced it five separate times: a mechanism that appears to validate something and does not. A manifest check that verified a run against its own output. A snapshot read but never compared. A reuse shortcut that skipped the write it was guarding. A `.gitignore` rule naming one directory when the next download used another. And this. None raised an error; all five reported success. The common shape is that the check and the thing being checked were allowed to come from the same source, so agreement was guaranteed rather than earned.

An undefined metric rendered as a real zero. The long-context arm showed citation precision `0.000` beside retrieval's `0.567`, reading as "long context cites badly". It *cannot* cite a gold chunk - its context is one whole-document pseudo-chunk. An empty gold list made precision compute as a genuine `0.0`. Now `None`, printed as "not measured".

14b. The artifact-provenance mistakes, in full

Everything in this project that failed silently failed the same way: an artifact computed under one set of assumptions was consumed under another, and nothing in between checked. Each instance below produced numbers rather than errors, which is why every one was caught by an audit rather than by a test.

A corpus whose definition was a query. The corpus was "the four most recent 10-K filings per company". That is not a corpus, it is a question whose answer changes whenever one of thirty companies files an annual report -- which Procter & Gamble duly did, so a rebuild produced its FY2026 filing and dropped the FY2022 one the gold labels point into. `ingest()` now rebuilds exactly the filings the committed manifest names. Worth noting why it never appeared locally: the EDGAR client caches submission listings, and this machine's copy predated the filing. A stale cache was hiding a reproducibility bug from the only machine that could have

noticed it early.

A judge asked to grade an answer against the words "(full document)". The long-context arm cites one `<doc>#fulldoc` pseudo-chunk that exists nowhere in the chunk map, and the faithfulness judge looked context ids up in that map with a string literal as the fallback. Every long-context answer would have been scored against a placeholder, and the verdict would have appeared in the faithfulness column as data. It was never caught because faithfulness had never finished running; it would have looked like a real measurement the first time it did.

Faithfulness was never blocked by quota, and I said it was three times. The judge loop sat inside the same `try` as answer generation, so a quota error while answering skipped judging entirely. The two use different models with separate allowances, and the judge's had capacity throughout -- it completed 432 calls elsewhere in this project. The evidence was in every run's usage block: nineteen live calls, none of them judge calls. "Quota exhausted" was true every time while being the wrong answer to why *this* metric was missing, and a true statement that answers a question nobody asked is a comfortable place to stop looking.

A guard that permitted the loss it existed to prevent. A quota-limited re-run overwrote a finished twelve-query generation result with a one-query one, so a guard was added to refuse a shrinking file. It compared the number of scored answers -- and faithfulness verdicts live *inside* those scores, so a re-run producing an identical count with every verdict null would have passed the check and destroyed the most expensive data in the file.

Scores matched to the wrong shortlist, twice. Cross-encoder scores are valid only for the shortlist they were computed over. The exporter skipped hybrid configurations on the grounds that dense vectors "did not exist yet" -- true when written, false ever since -- so `hybrid-plus-rerank` was measured on a BM25 shortlist while carrying a name that says otherwise. Then the consumer selected score files by whichever covered the most queries, a tiebreak that would have handed hybrid scores to every BM25 arm the moment a second file existed.

That row was withdrawn rather than footnoted, and measuring it properly settled the question: on its own shortlist it scores 0.1869, *below* both the baseline and the 0.2003 the withdrawn version reported. The wrong number was not merely unfounded, it was flattering.

A coverage check that compared two different things, in three files. Query vectors are returned keyed by query *text*, because that is what a retriever is handed. Comparing the size of that mapping against the *query* count reported 582 of 586 for a complete artifact, because a handful of queries word the same question. The same comparison was written independently in the runner and the exporter, so fixing one left the other wrong -- which is what happens when two places ask the same question of one artifact and each answers it separately.

A benchmark defect the miscount exposed. Chasing those four queries found that eight of them on the original wording, twelve paraphrased, share their exact text with another query that has *different* gold. A figure reported in two consecutive filings produces the same question twice, labelled against each, and both labels are correct. Every retriever sees one string and returns one ranking, so at most one of each pair can score. They are kept and reported on every run: the penalty falls on every configuration identically, so comparisons stay fair, and dropping them would have to happen across both wordings at once or the two runs would stop scoring the same queries.

A label that asserted something the weights do not have. The third embedding arm was called `finance-e5` and commented as domain-adapted to financial text. `intfloat/e5-base-v2` is trained on general web data like the rest of its family. The arm had never been measured so no number rested on it, but a name claiming a property a reader cannot check is its own kind of false result. It is now `e5-base-v2`, and the axis is what it always was: English-specialised against multilingual.

Growing the eval set would have erased the label audit. `write_eval_set` overwrites unconditionally and freshly built queries carry `GENERATED` with no checker fields, so re-running the builder to add queries would have replaced all 216 audited labels -- including the 44 rejections the robustness check rests on -- with unaudited ones of the same id. The file would have looked normal and merely larger.

Two ignore rules that were each correct and each too narrow. `.gitignore` named `results/retrieval-ablation/` after one 8.5 GB download; the next unpacked to `results/results/` and sat unignored. Broadening it to `results/*/` then swallowed the one directory worth committing -- the archive of completed runs against the earlier eval set.

14c. The mistake that kept recurring, and the one I made reporting it

Four times, prose in this document and the README contradicted the results files while the results themselves were correct. A grid re-run changes fourteen numbers; the tables quoting them are maintained by hand; nobody re-reads a document they did not just edit. The worst instance reported `retrieval-dense-bge` as significant at p = 0.0444 when the authoritative value was 0.059, and another described a configuration as *not* significant when it had become significantly worse.

Every one was caught by an audit script that walks every decimal figure in the documentation and checks it against the values the results files contain. Not one was caught by reading. That is the useful finding: at this density of numbers, review does not work and mechanisation does. **The fix is to generate these tables from `results/*.json` rather than transcribe them**, which `scripts/render_tables.py` now does: seven tables across this document and the README are rebuilt from the results files, and `--check` fails the test suite when a document disagrees with them. It found a real omission on its first run -- the hand-written overlap split was missing `hybrid-plus-rerank` and `retrieval-hybrid-rrf`, which are the top two configurations in that table.

That still leaves the *sentences*, which is where every one of these errors actually lived. `scripts/audit_figures.py` reads every figure quoted in prose and requires it to appear somewhere in `results/`; anything legitimately derived needs an entry with a written reason. It found three live errors the first time it ran, all in section 21 and Finding 2, and they are described in 14e.

The PDF renderer has a smaller version of the same idea -- it verifies that known strings survive into the rendered document -- and it earned its place late by failing on `0.1953`, the baseline nDCG at 216 queries, which is 0.1971 at 586. It was not detecting a broken PDF. It was detecting that the prose had stopped agreeing with the results.

And then that commit was pushed anyway. The gate printed its failure, the push went out, and a follow-up check caught it. A gate whose output nobody reads is decoration, which is every silent bug above wearing different clothes.

14d. Six defects the fix introduced, and what they have in common

Mechanising the tables was the right call and it broke the document immediately. All six failures below were found in the hour after the generator started working, and every one of them passed the checks that existed.

The check that could not fail. `--check` printed "no generated blocks" for a document with no markers and exited 0. Deleting the markers would have returned the tables to hand maintenance with the gate still green -- the precise failure the generator was written to prevent, reproduced inside the prevention. An unmarked document is unverified, not clean.

Markup printed into the PDF. The markers are HTML comments, invisible in every markdown viewer, so reading the source and reading the rendered markdown both showed nothing. The renderer printed them verbatim: six lines of `<!-- generated:... -->` in the published document, and its verification step said *verified*.

Why the verification step could not see it. Its three checks all asked whether something expected was present -- ink on every page, known strings surviving, replacement characters absent. None of these defects removed anything expected. Asking *what must be absent* is a different question and it needed its own check. That asymmetry is the transferable lesson here, more than any individual bug.

`nDCG@10` printed as `nDCG@1`. Column widths were allocated from character counts, but the header row is drawn in bold, and bold Helvetica is wider per character, so the header overflowed a column sized for the same number of regular-weight characters and was shaved by one. Not a clipped label: the name of a different metric, in a document whose subject is which metric said what. `delta vs base` lost its last character the same way. Both were fixed by measuring with `get_string_width` under the font each row actually uses, and truncation is now reported instead of accepted.

A significance tick that became a question mark. The tables mark significance with `*`, which Latin-1 cannot encode, so `sanitise` degraded it to `?`. The cell read `0.0014 ?` in a column headed `p (Holm)`, which reads as doubt about the number. It survived because the replacement-character check only flagged *runs* of two or more. One destroyed character is already a corrupted document, so unmapped characters are now collected and named rather than counted.

Then the new check failed on a correct document. It searched the rendered text for the pattern `<!--.*?-->`, and the section you are reading quotes the marker syntax in order to explain it. A pattern cannot distinguish a quotation from a leak. A false alarm costs the same credibility as a missed defect, because the next failure is the one that gets waved through -- and this project has already pushed a commit straight past a gate that was printing its failure. The fix was to stop pattern-matching and read the exact comment lines out of the source, so the question becomes "did any of *these* reach the page".

How all five of the above were actually found: by rasterising a page and looking at it. Not by a check, not by reading the markdown. The one instruction in the original brief that kept paying off was to render pages to images before declaring the document done.

One shape, three times. A markdown construct that spans two source lines has to be joined before its emphasis spans can be matched. That was fixed for paragraphs, and the fix was never extended to blockquotes or to multi-line list items -- so a two-line bullet emitted its continuation as a separate paragraph flush against the left margin, visibly outside the list it belonged to. Fixing a defect for one code path and leaving its siblings is its own category of mistake, and it is only visible if you look at the page.

Two more the new check then found, both older than the check. Rasterising one page to inspect the table led to noticing that inline code inside a bold span printed its backticks -- 22 of them, most in the module-by-module list where every entry is a bold filename in backticks, because span matching only went one level deep. And the statement this document calls its central design decision, set as a two-line blockquote, printed its `**` markers as text: blockquotes were written line by line, so the paragraph-joining fix that solved straddling emphasis for ordinary paragraphs was never extended to them. Neither defect was introduced by the table generator; they were found because a check finally asked what should not be on the page, and the answer was "24 characters of markup".

And a crash in the cosmetic path. `label_for` called `Path.relative_to`, which raises on any path outside the repository, so a caller passing an absolute path from elsewhere got a `ValueError` instead of a label. Found by the first test that used `tmp_path`, which is the argument for writing tests that do not run inside the happy path.

`scripts/render_pdf.py` had produced six rendering defects across this project and had **no tests at all**. It has them now. That gap existed because the script is "just tooling", and tooling is exactly where an unverified failure gets published under your name.

14e. Three bugs found by re-running the arm that had been stuck

The generation arm sat at twelve queries for weeks, recorded as "quota-bound". That was true and it was also hiding three defects, none of which a re-run was expected to find.

The two arms shared one sample size. A long-context answer costs about 131,000 prompt tokens; a retrieval answer costs 7,300. Tying both to `--n-queries` meant the cheap arm was capped by the expensive one, so a day's allowance bought twelve queries of each rather than many of one. Splitting the budget took the retrieval arm from 12 answers to 66 across two days, and faithfulness verdicts from 5 to 32. The blocker was a design decision wearing a quota's clothing.

Growing the sample threw away everything already paid for. The sample was a function of `n`, so asking for 40 after a 12-query run drew a different twelve among them -- measured: only 4 of the original 12 survived. Every other answer would have missed the cache and been bought again. Pinning the previously answered ids makes a larger run a superset of the smaller one.

`from_cache` was False on every answer this project ever recorded. It was inferred as "the response carries no latency", but the client writes the measured latency *into* the cached body, so a cache hit always has one and the flag could never be true. Nothing depended on it until latency statistics did, and then the consequence was immediate: a run whose long-context answers all came from cache and whose retrieval answers were made live during a throttled window reported the long-context arm as **2.5x faster than retrieval** . Both numbers were real. The comparison was between a quiet session and a congested one, and it would have been published as a measurement.

The correction is in section 17 and it costs information: latency now reads *not measured*, because this run made no live calls at all. That is worse to look at and better to trust. The last honest measurement is kept in `results/archive/` rather than deleted, since it was a real same-session comparison -- and it says 3.7x, where the document had been claiming 1.9x by quoting the run's overall p95 as though it were one arm's.

Then the check found a whole table. Extending the figure audit to percentages -- because a percentage is derived from two values and moves whenever either does -- turned up a second, hand-maintained copy of the overlap-split table in this document, still carrying the 216-query numbers: the baseline at 0.1091 and reranking at +111.7%, forty lines from the generated table showing 0.0823 and +104.4%. Its decimals passed the audit, because `results/archive/` legitimately still contains them. Only the percentages exposed it. The table is now a generated block, and the same sentence's claim that "BM25 loses 76% of its score" was 73% in the README and 73.2% in the results.

The passage under it made a claim that had simply stopped being true: dense retrieval was described as the only configuration whose low-overlap score exceeded its high-overlap score. On the 586-query set no configuration does. The underlying point survives in the form the data supports -- the baseline gains 2.89x from a question quoting the document, reranking 1.31x, the embedding arms 1.16x -- but the sentence as written was false, and it had been read past many times.

A verification pass over every component, and what it found.

Asked to verify the whole project rather than trust it, the checks that mattered were the ones that recompute rather than re-read. The corpus verifies against its manifest; all 586 gold spans resolve and contain the value their query asks for; both wordings carry identical ids and gold; no chunker produces a duplicate id or a chunk

whose span does not slice to its own text; every recorded chunk count rebuilds exactly, including the 40,155 of the re-parsed row-sentence rendering, which is the axis that was once silently inert. The baseline nDCG@10 recomputed from scratch -- fresh corpus, fresh chunking, fresh index, fresh qrels -- comes to 0.1903 against a recorded 0.1903. Every Holm-corrected p-value recomputes to the stored value. And re-running the entire fifteen-configuration grid -- on *both* wordings, including the paraphrased one the headline finding rests on -- reproduces every metric and all fourteen significance verdicts exactly. The sole difference in either file is the wall-clock `seconds` field, which is what §"Reproducibility" already promises.

The last modules verified were the ones the suite covered least. `to_row_sentences` does what §8 says: each data row becomes a self-contained line carrying its column header and row label, so `31,370` arrives as "Research and development -- 2024: 31,370" rather than as a bare number three columns into row nine. The lenient-labels arithmetic in §13 reproduces exactly -- all six numbers of its worked example, and every figure in `results/unjudged_analysis.json`.

Two things came out of that. The unjudged analysis was run on the 216-query eval set and has never been re-run at 586, which neither §13 nor the limitations list said. That is the third measurement in this project cited without its scope after the set grew -- after the label audit and the generation sample -- and the pattern is worth naming: a figure that was complete when written becomes a *partial* figure the moment the thing it measured got bigger, without anybody editing a word.

And the SEC client's rate limiter had no tests at all. It is the one piece of code here whose failure is not a wrong number: exceeding the published ceiling gets the address blocked, and the corpus becomes unrebuildable for anyone reading this repository. It works -- five requests at 20/s take the four intervals between them, a non-positive rate is refused, and the configured 5/s is half SEC's published 10 -- but "it works" was not previously demonstrated anywhere. It is now.

Verifying the retrieval internals by hand found nothing wrong with them. BM25 reproduces a hand-computed score from the standard formula exactly, its IDF floor holds, and its tokeniser emits both `34,550` and `34550` so a query written either way still matches. RRF matches a hand-computed reciprocal-rank sum. `lexical_overlap` recomputes to the stored value for all 1,172 queries across both wordings, which is what the low/high split in Finding 2 rests on.

One tokenisation quirk turned up and was dismissed with evidence rather than argument: `&` splits, so "AT&T" becomes the tokens `at` and `t` and the company name stops discriminating. That looks like it should hurt, and it does not -- those 30 queries average nDCG 0.277 against 0.171 for queries with no ampersand at all.

A stale figure the audit could not see, and the fix that did not work. The README described the eval set's median content-word overlap as 0.46 with a range of 0.22-0.88. The median is 0.50, the mean is 0.47, and the range is 0.17-0.82 -- so the number was stale *and* mislabelled, since 0.46 was the mean of the older 216-query set. Two decimal places, which is below what the figure audit scans, so nothing caught it.

Adding a two-decimal pattern was the obvious response and it is decoration. The results files hold thousands of numbers, and 91 of the 101 values between 0.00 and 1.00 already appear among them -- "0.46" included. The extension passed on the exact error that motivated it. It was measured, found vacuous, and removed rather than shipped, which is the first time in this project a guard was caught before rather than after it went in.

One latent defect was found in code rather than prose. `DenseIndex` normalises its passage matrix defensively, with a comment explaining that a non-unit vector "turns the dot product into something that is not cosine similarity -- silently, since the ranking still looks plausible". The query side then *asserted* that same property in a comment instead of enforcing it. Every embedder this project uses returns unit vectors -- checked against the committed artifacts, where every stored norm is exactly 1.000000 -- so every published dense score is genuine cosine and no measurement moved. But the guarantee was one-sided, and a caller supplying an unnormalised embedder would have got scores scaled by the query's magnitude while the class documented cosine. Ranking would have survived, since a positive constant does not reorder; anything

reading the scores would not. Now normalised on both sides, with a test that supplies a deliberately unnormalised embedder and requires the scores to equal hand-computed cosine.

One documented claim was false. Both documents said the significance picture was unchanged between the full label set and the audited subset. `chunk-fixed256o32` is p = 0.063 on all labels and p = 0.046 on the accepted subset, so it crosses 0.05 -- the sentence was true when written and was never re-checked after the eval set grew from 216 queries to 586. It is corrected above.

Three things were found that are not defects but belong in an honest account. Ten of the 120 filings -- Intel's four, GE's four, Duke's two -- carry no Part headings at all, because those companies use a cross-reference index instead of Part/Item section divisions, so their section paths are shallower than the rest of the corpus. The service timings quoted here are single-run figures on one machine and vary by about 20% between runs, which is normal and is not the same kind of number as a metric. And `Embedder.encode` claims a caller "cannot forget" the query/passage distinction while defaulting to passage; every caller in this repository uses the explicit wrappers, so the claim is stronger than the code but nothing relies on it.

The guard was right and I went around it anyway.

Judging the long-context arm needs no new answers, so every answer comes from cache and no call is timed. `publish` refuses that write, because live latency samples are the one field a successful-looking cached re-run destroys. The refusal was correct. Its advice was not: it said to archive the file and then *delete* it to let the run through, and I followed my own instruction.

Deleting the results file does let the write through. It also erases the resume state, because `previously_answered` reads the pinned query ids out of that same file. With nothing to pin, the run re-drew its sample from scratch, answered thirteen fresh queries instead of reusing sixty-six, spent the day's request allowance doing it, and published a file holding a third of the measurements. The guard that exists to refuse exactly that write could not, because the advice had removed the thing it guards against.

Two lessons, and the second is the useful one. Overriding a guard should never require destroying state -- `--force-publish` now does it explicitly, and the advice points there instead. But the deeper point is that the failure came from a piece of writing rather than a piece of code: the guard, the pinning and the archive were all correct and had all been tested. What was wrong was a sentence telling someone what to do next, and no test covers a sentence. That is the same category as the docstrings elsewhere in this section describing behaviour their functions never had, and it is the category this project has been worst at.

The run was not wasted, and it was not as recoverable as I first said either. It produced the first long-context faithfulness verdicts this project has ever had -- nine answers judged, all nine faithful -- which is the measurement §17 has been reporting as absent. I then wrote that restoring the larger file and re-running would merge them in through the ordinary code path, because the judgements are cached.

That was wrong for the same reason the original loss happened, one step further on. Losing the pinning re-drew the *long-context* sample too, so those nine verdicts describe eleven queries that share exactly one member with the published run's eleven. The cache holds them; nothing in the published sample can use them. The re-run had ten fresh judge calls to make and met the day's limit first.

So that figure was archived as `results/archive/generation-long-context-faithfulness.json`, labelled as what it is: a measurement of a different eleven queries, which cannot be paired with the published accuracy and cost numbers without comparing two samples as though they were one.

It was then done properly. With the results file left in place, the pinning intact and `--force-publish` used instead of deleting anything, the eleven long-context answers §17 actually tabulates were judged: **0.955**, against retrieval's 1.000. The archived run stays, because it is what the detour produced and deleting it would tidy away the evidence for this whole passage.

Twice now I have said "this is recoverable" before checking which queries were involved. The first time cost a day's quota. The second cost only a paragraph, and only because the paragraph was checked before anyone read it.

And then the same class of ambiguity in the reader it was built for. Dry-running the finished workflow against the *real* sample file rather than a fixture -- seven entries marked, five accepted, two rejected -- produced the message "6 changed". Both numbers were correct. One of the two rejections landed on a query the model audit had already rejected, so applying it changed nothing.

But nothing in the output said so, and the person who ticked seven boxes has no way to tell agreement from a dropped verdict. That is the project's own "not measured versus measured zero" rule appearing in a new place: here it is "changed versus already consistent", and collapsing them makes a complete run look like it lost an answer. `apply_verdicts` now returns both counts and the CLI reconciles them against what was marked.

Worth noting how it was found. The unit tests were green and remain green -- they use fixtures where nothing is pre-rejected, so the case could not arise. It took running the real file, with the real 44 model-rejected queries in it, to produce the condition at all.

A mechanism the documentation described and the code did not have. `build.py` writes `verification_sample.md` for a person to mark, and its docstring said "a reader marks each entry, the marks are fed back, and the verified subset becomes reportable on its own terms". Nothing fed them back. There was no code that could read a tick, so the file could have been filled in completely and the project would still have reported its labels as unverified.

That is the `latency_stats` defect again -- a docstring describing behaviour its function never performed -- and the cost here is somebody's afternoon. The reader exists now, and it refuses the obvious temptation: an unmarked sample reports the rejection rate as *not measured* rather than as zero, a partly marked one reports a rate over the marked subset and says so in the same sentence, and an entry with both boxes ticked is skipped rather than resolved in whichever direction seemed likely. Applying the verdicts is a separate flag, because reading a file and changing a benchmark are different acts, and only the queries a person actually marked are touched -- treating "absent from the sample" as a verdict would relabel 546 of 586 queries from one afternoon's work.

And CI had never passed. Asked what was left, the honest answer included "I have not seen the workflow run", and checking found the badge reading *failing* -- not from a recent change, but across every run in the repository's history. The durations gave it away before the badge did: nine to fourteen seconds, for a job that is supposed to install a project, run five hundred tests on two Python versions, and render a PDF.

Two causes, both the same shape. The matrix declared `python-version: ["3.12", "3.14"]` and nothing consumed it -- there was no `setup-python` step and the value was never passed to `setup-uv`, so the version list was decoration. And the install step used `uv pip install --system` while every later step ran `uv run --no-project`, which does not see what `--system` installed. Locally both work, because a project virtualenv exists here for `uv run` to find; on a clean runner neither does. The fix is `uv sync --extra dev --extra service` and plain `uv run`, which is what the lockfile is for.

The lesson is the one this document keeps arriving at from different directions. A green checkmark nobody looked at is worth exactly as much as a gate whose failure nobody read, and this repository had been carrying both while its author -- me -- described the suite as passing. It was: locally, in an environment the workflow never reproduced.

The guard was extended again, for the field it had just failed to protect. `publish` counted scored answers and faithfulness verdicts per arm. It did not count live latency samples, and latency is the one figure here that a cached re-run destroys while looking entirely successful: every answer comes back from disk, the

score count matches, the verdicts match, and no call is timed. Replaying the real files through the old guard confirms it -- the run that replaced a valid 11-and-10 same-session comparison with "not measured" was allowed through, and the new one refuses it.

Refusing alone would have been the wrong fix. That run also gained sixteen scores and seven verdicts, so "delete the file to replace it deliberately" would have thrown away the gain to keep the smaller thing. When latency is the only field that regressed the message now says to archive first, which is what was actually done here by hand.

And the limitations list was the most wrong thing in the document. Asked whether the project was finished, the honest answer needed checking rather than recalling, and checking found that four of the nine items in section 21 were false. It claimed six of fifteen configurations were unmeasured when all fifteen had been measured on both wordings; it called query paraphrasing "not done" when paraphrasing is the headline finding; it reported generation on 12 of 216 queries; and it said faithfulness was "not measured at all" when twelve verdicts existed. A paragraph in Finding 2 was worse than stale -- it claimed dense retrieval beat the baseline on low-overlap queries, quoting figures from the 216-query set, while a generated table forty lines above it showed dense losing by 17.8%. The prose contradicted a table on the same page.

A limitations section that understates the work is not more honest than one that overstates it, and it is the section a reader trusts most. The lesson is narrow and worth stating: generating the tables moved the drift into the sentences around them rather than removing it, and only a check that reads the sentences finds that.

And the guard against losing data had the same shape as the bug it prevented. `publish` refuses to overwrite a file holding more scored answers or more faithfulness verdicts than the current run produced. It compared *totals*. A `--skip-long-context` run of 40 retrieval queries has more scores than a file holding 11 retrieval and 10 long-context, so the totals said "more" and the write would have gone through, taking every long-context measurement with it. Verified by running the old comparison against the real files: it wrote, and all 10 long-context scores were gone. The guard now compares per arm. Its own docstring already explained that comparing only totals is how verdicts get destroyed inside a matching score count -- the same reasoning, one level up, left unapplied.

Part IV - What was measured

15. The retrieval ablation

Full corpus, 390 of 586 queries judgeable by every configuration. **Original wording.** Chunk counts differ by chunker and that is the point of four of these rows: 42,215 under structure-aware splitting, 37,498 for the fixed-512 baseline, 75,084 at fixed-256, 29,556 semantic. Quoting one figure for the whole table would misstate three of them.

configuration	nDCG@10	95% CI	Recall@50	MRR	delta vs base	p (Holm)
rerank-candidates-50	0.2198	[0.186, 0.255]	0.5667	0.1891	+0.0227	1.000
chunk-semantic95	0.2189	[0.187, 0.252]	0.6641	0.1828	+0.0218	0.787
chunk-struct512	0.2135	[0.180, 0.249]	0.5667	0.1905	+0.0164	1.000
retrieval-bm25-struct	0.2135	[0.180, 0.249]	0.5667	0.1905	+0.0164	1.000
rerank-candidates-25	0.2116	[0.179, 0.245]	0.5667	0.1843	+0.0145	1.000
rerank-bm25-100	0.2068	[0.174, 0.241]	0.5487	0.1797	+0.0097	1.000
baseline-bm25-fixed512	0.1971	[0.166, 0.230]	0.5385	0.1769	-	-
rerank-candidates-200	0.1924	[0.161, 0.226]	0.5051	0.1674	-0.0047	1.000
hybrid-plus-rerank	0.1869	[0.155, 0.220]	0.4744	0.1626	-0.0102	1.000
retrieval-hybrid-rrf	0.1817	[0.150, 0.215]	0.5205	0.1630	-0.0154	1.000
tables-row-sentences	0.1644	[0.135, 0.195]	0.5324	0.1501	-0.0327	0.297
chunk-fixed256o32	0.1556	[0.127, 0.185]	0.4474	0.1376	-0.0415	0.063
retrieval-dense-bge	0.1044	[0.079, 0.132]	0.2718	0.0928	-0.0927	0.0014 *
embed-e5-base-v2	0.1007	[0.077, 0.127]	0.2949	0.0853	-0.0965	0.0014 *
embed-e5-base	0.0367	[0.023, 0.052]	0.1795	0.0312	-0.1604	0.0014 *

Nothing beats the baseline significantly here. The three significant rows are the dense arms, all significantly *worse*.

Finding 0 - the benchmark was hiding the effect it existed to measure

This is the most important result in the project, and it was invisible until the last thing on the list got done.

Every query in this benchmark was generated from a table row and reused that row's label word for word. Paraphrasing rewrites the questions the way a person would ask them, touching nothing else - same corpus, same gold spans, same query ids, same 390 shared queries. Then the grid runs again.

configuration	original	paraphrased	change	delta vs base	p (Holm)
hybrid-plus-rerank	0.1869	0.1208	-35%	+0.0680	0.0014 *
rerank-bm25-100	0.2068	0.1149	-44%	+0.0621	0.0014 *
rerank-candidates-200	0.1924	0.1109	-42%	+0.0581	0.0014 *
retrieval-hybrid-rrf	0.1817	0.1022	-44%	+0.0494	0.0014 *
rerank-candidates-50	0.2198	0.1004	-54%	+0.0476	0.0014 *
rerank-candidates-25	0.2116	0.0975	-54%	+0.0447	0.0016 *

retrieval-dense-bge	0.1044	0.0839	-20%	+0.0311	0.0399 *
chunk-semantic95	0.2189	0.0663	-70%	+0.0134	0.648
chunk-struct512	0.2135	0.0643	-70%	+0.0114	0.648
retrieval-bm25-struct	0.2135	0.0643	-70%	+0.0114	0.648
tables-row-sentences	0.1644	0.0597	-64%	+0.0069	0.829
embed-e5-base-v2	0.1007	0.0506	-50%	-0.0022	0.841
chunk-fixed256o32	0.1556	0.0407	-74%	-0.0121	0.648
embed-e5-base	0.0367	0.0137	-63%	-0.0391	0.0014 * (worse)
baseline-bm25-fixed512	0.1971	0.0528	-73%	-	-

On the original wording not one configuration beats the baseline significantly. On the paraphrased wording seven do - both hybrid arms, every reranking arm, and dense retrieval. The study's entire conclusion was a property of how its questions happened to be phrased.

Two configurations reverse sign outright. `hybrid-plus-rerank` sits *below* the baseline on the original questions at 0.1869 and is the best configuration in the study on the paraphrased ones. `retrieval-dense-bge` is significantly *worse* than BM25 on one wording (-0.0927) and significantly *better* on the other (+0.0311) - same retriever, same corpus, same labels, opposite verdicts at the same confidence.

Four, and not the seven an earlier version of this document reported. Completing the grid is what changed it, and the way it changed is worth more than the number:

configuration	delta	p (raw)	Holm over 12	Holm over 14
retrieval-dense-bge	+0.0516	0.0074	0.044 *	0.059 x
rerank-candidates-25	+0.0488	0.0056	0.039 *	0.050 x

Neither was re-run. Neither result moved by a thousandth. They stopped being significant because two *unrelated* configurations - semantic chunking and a second E5 model - were measured, enlarging the family the correction is applied over. "Significant" turned out not to be a property of `retrieval-dense-bge` at all; it was a property of `retrieval-dense-bge` *and the contents of the grid*, and the grid's contents are a decision somebody made in `configs.py`.

This is the correction doing its job, not failing. Testing more hypotheses against the same 143 queries really does make it likelier that one of your successes is noise, and Holm charges for that honestly. But it exposes something the significance-testing framing tends to hide: the threshold is a property of the experiment as a whole, so a result can be lost by measuring something else entirely. The defensible reading is that both sit on the boundary - 0.050 and 0.059 are not meaningfully different from 0.05, and calling one a finding and the other nothing would be reading the third decimal place as if it meant something.

The `change` column is the mechanism. BM25 loses 73% of its score once questions stop quoting their answers; dense loses 17%, because it was never using the overlap. About three quarters of the baseline's score was the benchmark handing it back its own words - and every semantic method was being compared against that.

The candidate-depth ordering **inverts** too: depth 200 was the worst reranking configuration on the original wording and is the best here, which is what a reranker doing real work should do with a deeper shortlist. On the original queries a deeper shortlist was only more chances to demote a hit BM25 had already placed correctly.

One thing does not move: chunking and table rendering stay null on both wordings. The confound was specific, not a haze over everything, and that is precisely why it survived so long. Most of the table looked stable.

The middle column is the mechanism. BM25 loses 73% of its score once the questions stop quoting their answers; dense loses 17%. Dense was never using the overlap, so it had almost nothing to lose. About three quarters of what BM25 was scoring was the benchmark handing it back its own words.

Two earlier conclusions dissolve here, and both had been written up as findings. `embed-e5-base` is significantly *below* baseline on both wordings -- the one configuration the study can say is simply worse. Meanwhile the chunking and table-rendering axes stay null on both wordings - which matters, because it shows the confound was specific rather than a haze that moved every number. A benchmark can be badly wrong about one comparison and perfectly fine about another, which is exactly what makes this kind of flaw hard to notice: most of the table looks stable.

A first version of this section reported the reranking result before it could be measured, and retracting it is more instructive than the claim was.

The direction it claimed turned out to be right - every reranking arm is significant above. That is not a vindication, it is the uncomfortable part. The claim was unfounded when it was made, and being lucky about a conclusion is indistinguishable, at the moment of writing, from being right about it.

The reranking and dense arms are served from precomputed GPU artifacts keyed by query id. Paraphrasing keeps query ids deliberately - that is what makes the two eval sets comparable. So the paraphrased run reused cross-encoder scores and query vectors computed from the *original* wording: the reranker was scoring `(original question, passage)` pairs whose questions quote the answer, while being credited for ranking paraphrased ones. The measured "improvement" was substantially the confound leaking back in through the artifact.

Nothing raised. Coverage was complete. The p-values were small, correctly computed, and correctly corrected. What made the number attractive - a large effect, appearing exactly where the hypothesis predicted, after a deliberate intervention - is precisely what should have made it suspicious. It confirmed what was expected, and that is when a result gets the least scrutiny and deserves the most.

The lesson generalises past retrieval. Every piece of statistical machinery in this project was correct: the paired test, the bootstrap intervals, the Holm correction. All of it was applied conscientiously to numbers produced by a benchmark that favoured one arm, and none of it could have noticed, because none of it was wrong. Significance testing protects against reading noise as signal. It offers no protection whatsoever against a well-measured answer to the wrong question. The only thing that caught this was recording lexical overlap per query - deciding, before any of the results existed, to measure the confound rather than argue about it.

Finding 1 - nothing was shown to help; one thing was shown to hurt

Reranking takes the top four slots, and **no improvement survives Holm correction** - the best configuration's corrected p is 1.000. At 143 queries the CI spans about +/-0.055, so a 0.019 gap is inside the noise.

Writing "reranking lifted nDCG@10 from 0.195 to 0.215" would be correct arithmetic and a false finding.

Exactly one comparison in the grid does survive: `embed-e5-base` at 0.1540 *below* baseline, p = 0.001. That asymmetry is worth sitting with. The study was built to detect improvements and detected none; the only thing it could resolve at this sample size was a configuration failing badly. Significance is a statement about effect size relative to noise, not about importance, and a benchmark too small to confirm the effect you are hoping for is still perfectly capable of confirming one you are not.

A result that large invites a bug hypothesis, and E5 has an obvious one: it requires literal `query:` and `passage:` prefixes and loses a lot of accuracy without them. The prefixes were verified present in the code path, and then the claim was checked rather than trusted, by asking where the gold passages actually land. Random ranking among 42,215 chunks would put gold in the top 1,000 for about 2.4% of queries; E5

manages 59.3%, against BGE-M3's 71.8%. E5 is working. It is just worse here - and the next section explains most of what "here" is doing.

Finding 2 - the average hides a large real effect

Splitting queries at 0.4 content-word overlap with their gold passage:

configuration	low-overlap nDCG	vs base	high-overlap nDCG	vs base
baseline-bm25-fixed512	0.0823	-	0.2378	-
rerank-bm25-100	0.1682	+104.4%	0.2205	-7.3%
rerank-candidates-50	0.1680	+104.1%	0.2382	+0.2%
rerank-candidates-25	0.1664	+102.3%	0.2276	-4.3%
rerank-candidates-200	0.1564	+90.1%	0.2051	-13.7%
hybrid-plus-rerank	0.1604	+95.0%	0.1963	-17.5%
retrieval-hybrid-rrf	0.0921	+11.9%	0.2134	-10.3%
retrieval-dense-bge	0.0676	-17.8%	0.1174	-50.6%

The cross-encoder roughly doubles performance where the question does not share wording with its answer, and slightly hurts where it does. Exactly the right shape: where BM25 already has an exact string match there is nothing to fix, and reordering can only push a correct top hit down. The +0.0097 average is a large positive on half the queries cancelled by a small negative on the other half.

The dense rows make the same point from the other direction, and they are the reason the overall dense numbers should not be read as "embeddings are bad at finance". Read the ratio of each configuration's high-overlap score to its low-overlap one - how much it gains from the question quoting the document. The baseline gains the most, at 2.89x. Reranking cuts that to 1.31x. The embedding arms sit lowest at 1.16x, and dense at 1.74x. That is the signature of a method that does not match on strings: it benefits least when the query reuses the document's wording, which is also why it looks worst on a benchmark built out of that wording.

(This passage previously said dense was the only configuration whose low-overlap score exceeded its high-overlap score, quoting 0.1346 against 0.1143. On the 586-query set no configuration does - dense scores 0.0676 low against 0.1174 high. The claim was true of the smaller benchmark and was never re-checked. The ratio form above survives the change because it was the actual point.)

Which means the benchmark is not neutral between the two families. Its queries are generated from table rows and reuse the row labels verbatim, handing BM25 an exact match on most of them. On the low-overlap subset - the queries that look more like something a person would type - the gap narrows sharply: `retrieval-hybrid-rrf` goes from 10.3% *behind* the baseline on high-overlap queries to 11.9% ahead on low-overlap ones, and reranking roughly doubles. The defensible claim is "BM25 wins on queries phrased in the filing's own words", not "BM25 wins on SEC filings", and the difference between those two sentences is the entire value of having measured the split.

This was the clearest argument for query paraphrasing, and it is the reason paraphrasing was built. A benchmark that systematically advantages one arm will report that arm winning, and will do so with tight confidence intervals and a straight face. Rewording the questions and re-running the whole grid is what turned that argument into the measurement in Finding 0.

(An earlier version of this paragraph claimed dense retrieval beat the baseline on the low-overlap subset, quoting 0.1346 against 0.1091. Those were figures from the 216-query eval set, left in place after the set grew to 586. On the current data dense is 0.0676 against the baseline's 0.0823 - it loses by 17.8%, as the generated table above states forty lines earlier. The paragraph contradicted a table on the same page, which

is what hand-maintained prose does eventually.)

Finding 3 - deeper shortlists are worse

depth	nDCG@10	recall ceiling
25	0.2116	46.8%
50	0.2198	56.3%
100	0.2068	64.2%
200	0.1924	73.7%

Depth 200 has by far the **best** ceiling and the **worst** score - below no reranking at all. The cross-encoder is not failing to *see* the answer; given more candidates it actively promotes wrong ones past it. The intuitive tuning move - widen the shortlist - measurably backfires.

16. Label quality, and whether the conclusions survive it

All 216 labels were audited by a language model asked to reject on specific checkable grounds. **44 of 216 rejected - 20.4%**, 0 unparseable. Rejections concentrate on entity/place names used as subjects ("united states", "duke energy ohio") and on passages with several figures under one label.

Re-running the whole grid on the 542 accepted labels:

- **Every configuration moves by less than 0.011** -- from -0.0015 for `embed-e5-base` to +0.0101 for `rerank-candidates-50`. The rejected labels were penalising systems roughly equally, which is what you would expect if they were unanswerable rather than wrong in some direction.
- **The ranking is stable, not identical.** Adjacent configurations separated by thousandths trade places, which is noise and is exactly why none of them is reported as a finding.
- **The significance picture survives, with one exception worth naming.** `chunk-fixed256o32` is p = 0.063 on all labels and p = 0.046 on the accepted subset: it crosses 0.05 when 33 rejected labels are removed. Nothing else changes verdict, and no conclusion here rests on that row -- it loses to the baseline on both label sets, by 0.0415 and 0.0443. But the honest reading is that a configuration balanced on the third decimal can be decided by the label set, which is precisely why this document reports intervals beside every point estimate.

This sentence previously read "the significance picture is unchanged". It was written when that was true and never re-checked against the accepted-subset run after the eval set grew, which is the same drift as every other stale claim in section 14 -- caught here only because verification asked the results rather than the prose.

The audit covered the original 216 queries. The 370 added when the set was extended are unaudited and carry `GENERATED`, so "accepted" means "not rejected by the audit that ran", not "checked". That distinction is why verification status is stored per query rather than asserted once for the project.

These are marked `MODEL_CHECKED`, never `HUMAN_VERIFIED`, and a test enforces it. A model auditing labels a program generated from the same tables is not an independent second opinion, and cannot see that a *different* passage would have been the better gold.

17. Retrieval versus long context

`gemini-3.6-flash`, costs computed from the API's own reported token counts. The two arms no longer share a sample size: one long-context answer costs about 131,000 prompt tokens against 7,300 for a retrieval answer, so tying them together capped the whole evaluation at what the expensive arm could afford in a day.

	retrieval (top-10)	long context (whole filing)	ratio
queries answered	110	11	-
mean prompt tokens	6,752	130,819	19.4x
cost per query	$0.010308	$0.196508	19.1x
accuracy, of answered	0.692	0.636	-
refused	58 of 110	0 of 11	-
gold reachable in what the arm read	110 of 110	11 of 11	-
faithfulness	0.981 (52 judged)	0.955 (11 judged)	-
p95 latency	11.698 s	not measured	not comparable

The long-context arm has a reachability ceiling, and it had never been reported. It is handed a prefix of the filing, capped at 800,000 characters. 20 of the 120 filings exceed that, the largest losing 40% of its text, and `stuff_document` labelled every one of them `#fulldoc` -- while its own docstring claimed the id recorded the truncation. Across the eval set, 32 of 586 queries have gold beyond the cut: the arm is handed text that cannot contain the answer and would be scored as though it had the whole filing, which is precisely the mistake the ablation avoids by reporting a reachability ceiling for every chunker.

All 11 queries in the table above are reachable, so nothing there is affected. The id now records truncation and `long_context_reachable` counts it, so the next run records the ceiling rather than leaving it to be discovered.

The brief's "roughly 1,250x cheaper" is not reproducible. 1,250x requires assuming a 1M-token context, an ~800-token retrieval prompt, *and* zero output cost. The measured ratio is in the table above.

Latency is reported as not measured, and that is a correction. The number previously printed here was wrong twice: it quoted the run's overall p95 as the long-context arm's, and the two arms' figures came from different sessions. The last run in which both arms made live calls in the same session gave retrieval p95 4.542 s against long-context 16.908 s -- a 3.7x ratio, not the 1.9x stated here for two commits. That run is kept as `results/archive/generation-n12-same-session-latency.json`, because re-measuring it costs a fresh long-context pass of roughly 1.4M prompt tokens, which is more than a day's free-tier allowance. Cost survives the same problem: token counts do not depend on when a call was made.

Retrieval refuses half the questions, and that is the whole story: it declined 58 of 110 because the answer was not in its top-10. On the ones it did answer it is now more accurate than long context, 0.692 against 0.636, and it cites its sources -- which the long-context arm structurally cannot, since its entire context is one document. So the honest summary is that retrieval is eighteen times cheaper and answers half as often, and the gap is created by its first stage, not by the model writing the answer. That is Part IV's finding reached from the other direction.

This paragraph used to say long context wins on accuracy. It did, at 11 and 27 answers. The ordering reversed once the retrieval arm reached 66, which is a useful demonstration of how little a two-decimal accuracy gap means at these sample sizes: neither arm has a confidence interval narrow enough to call this, and the refusal rate is the difference that survives.

Three caveats that cut against the result. Long context is **handed the correct filing** while retrieval must find it among 120. The samples are small and no longer equal -- 110 retrieval answers against 11 long-context ones -- because the arms were separated to stop the expensive one capping the cheap one. And faithfulness is now measured on both arms, on the same eleven long-context answers the rest of that column describes.

Long context is *behind*: 0.955 against retrieval's 0.981. One answer in each arm was judged less than fully grounded -- including one in the arm that was handed the correct filing outright, which is the failure mode people assume belongs to retrieval alone.

Retrieval's figure was 1.000 until the sample reached 89 answers, which is worth saying rather than quietly updating: a perfect score over 32 judgements meant "no counterexample yet", not "cannot happen".

That cell has now had two wrong explanations attached to it, and the second was mine from a few hours ago.

The first was "requires a paid tier", written before anything was measured. The second replaced it with arithmetic: a verdict costs about 131,000 prompt tokens because the judge must see the filing too, a day of this project has spent 2.1M prompt tokens, so about sixteen verdicts fit in a day -- and one verdict therefore costs what 18.3 retrieval answers cost. Every number in that is real. The reasoning is still wrong, because it assumes tokens are what runs out.

Reading the 429 body instead of guessing at it says otherwise. The server names the quota it enforced: `GenerateRequestsPerDayPerProjectPerModel-FreeTier`, with a value of 20. That is counted in **requests**, and against a request limit a 131,000-token judgement and a 7,000-token answer cost exactly the same. The eighteen-to-one trade-off I had just written down does not exist.

And the picture still does not close. The same day's run recorded 109 live calls against a limit the server reports as 20. Both of those are observations, and they cannot both be the whole story, so the constraint is not characterised here and this document will not pretend otherwise. What is established is narrower and still useful: the enforced limit is expressed in requests, so reasoning about token cost does not predict what a day buys, and the client now quotes the quota the server names instead of inferring one.

The general lesson is the one this section keeps producing. The first explanation was an assumption. The second was an assumption wearing arithmetic, which is harder to notice and took a deliberate look at an error body to dislodge.

18. Operating findings

GPU throughput, Tesla T4, 42,215 chunks each: BGE-M3 13.9 min (51 chunks/sec); multilingual-E5-base 4.8 min (149/sec).

Gemini free tier, measured not assumed: `batchEmbedContents` caps at **32** texts (64 -> `RESOURCE_EXHAUSTED`), roughly 3 requests/min/model - about **96 texts/minute**, or 7.3 hours for one embedding pass. Against Kaggle's ~3,060/min that is **32x**, which settled where GPU work belongs.

Thinking models consume the output budget. `gemini-3.6-flash` at `maxOutputTokens=16` returns **empty text** - 13 tokens went to `thoughtsTokenCount`. At 512 the same prompt answers correctly after 81 thought tokens. `thinkingBudget: 0` is rejected with HTTP 400.

Service: index builds in 31.7 s over 42,215 chunks; `/search` returns in **1.9 ms**.

Part V - Running and extending

19. Running it

```
uv venv && uv pip install -e ".[dev,service]"
uv run python -m ruff check . && uv run python -m pytest
```

422 tests pass offline with no API key and no model download.

Rebuild everything:

```
uv run python -m retrieval_ablation.corpus.ingest      # ~40 min, 120 filings
uv run python -m retrieval_ablation.evalset.build      # deterministic
uv run python -m retrieval_ablation.ablation.runner    # the ablation
```

GPU arms run on Kaggle (`notebooks/kaggle_gpu_arms.py`), because PyTorch cannot load on the development machine under an enforced Windows code-integrity policy - `WinError 4551`, and the log names `torch_cpu.dll`, so the CPU build is blocked too. Disabling that control is machine-wide and cannot be undone without reinstalling Windows.

20. Extending it

A new chunker: subclass `Chunker`, return chunks whose spans index the canonical text. No relabelling is needed - that is the payoff of §7.

A new retriever: implement `Retriever.search`. Return an empty list rather than padding with zero-scoring results.

A new metric: add to `metrics/retrieval.py`. Return `None` when it cannot be computed; never a plausible default.

A new ablation row: add a `Config` to `build_grid()`. A check asserts every non-interaction row differs from its reference on exactly one axis.

21. Honest limitations

1. **Queries were templated**, reusing the corpus's own wording - median overlap 0.50, mean 0.47. This is the limitation the paraphrased set was built to remove, and doing so produced the study's headline finding, so it is no longer a caveat but a measured axis. The original wording is still reported beside it because the gap between them *is* the result.
2. **Labels are model-checked, not human-verified**, and only 216 of the 586 were checked at all. The 370 added when the set was extended are unaudited.
3. **Single gold passage per query** understates retrieval by ~12% (§13), measured on the 216-query set and not re-run at 586.
4. **All fifteen configurations are measured**, on both wordings. This item used to read "six of fifteen unmeasured" and stayed in the list for several commits after the GPU runs closed that gap - a limitations

section that understated the work, which is the same drift as one that overstates it and no more honest.

5. **Generation is measured on 110 retrieval answers and 11 long-context ones**, of 586 queries, and the two arms no longer share a sample size. Cost ratios are solid. Accuracy is indicative at this sample. Latency is not measured at all in the published run; §17 says why.

6. **Faithfulness is measured on both arms** - 52 retrieval verdicts at 0.981, and 11 long-context verdicts at 0.955. Both are too few to quote as a rate. The long-context arm was the expensive one to judge, at about 131,000 prompt tokens per verdict, which is why it came last rather than not at all.

7. **Approximate token counting** (characters ÷ 4) rather than a real tokenizer. Every configuration uses the same counter, so comparisons are valid, but boundaries differ from a model's own.

8. **One-axis-at-a-time design cannot detect interactions.** One crossed cell is run explicitly.

9. **Absolute numbers are not comparable to published benchmarks** - this corpus is deliberately adversarial.

Glossary

BM25 - lexical ranking function combining rare-word weighting, term-frequency saturation, and length normalisation.

Bi-encoder - model embedding query and passage separately, compared at the end. Fast, precomputable, cannot model interaction.

Chunk - a retrievable unit, a few hundred words, carved from a document.

CIK - SEC's numeric identifier for a filing entity.

Cross-encoder - model reading query and passage jointly and scoring the pair. Accurate, expensive, usable only over a shortlist.

DCG / IDCG / nDCG - Discounted Cumulative Gain; the Ideal DCG of a perfect ranking; their ratio. See §4 for the IDCG trap.

Embedding - a list of numbers representing meaning, such that similar texts get similar lists.

Gold passage - the passage labelled as containing a query's answer.

Holm-Bonferroni - a correction controlling the family-wise error rate across many comparisons; uniformly more powerful than plain Bonferroni.

Inline XBRL - machine-readable financial tags embedded in filing HTML.

MRR - Mean Reciprocal Rank; the average of 1/(rank of first relevant hit).

Pooling bias - the distortion from metrics treating unjudged documents as non-relevant.

qrels - query relevance judgements: which passages are relevant to which query, at what grade.

Reachability - the fraction of gold passages a chunking configuration can represent at all.

Recall ceiling - the first stage's recall at the reranking shortlist depth; the hard upper bound on what reranking can achieve.

RRF - Reciprocal Rank Fusion; combines rankings by position, avoiding incomparable scores.

Span - a half-open character interval `[start, end)` into a document's canonical text.

10-K - a US company's annual report to the SEC.